

# Computer Organization

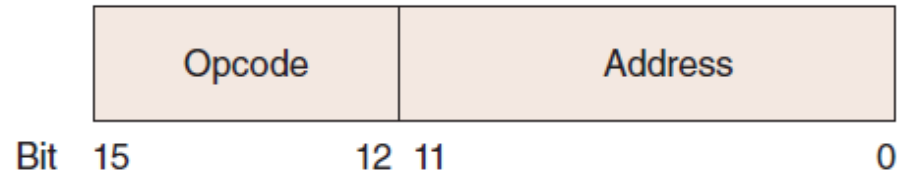
A CLOSER LOOK AT INSTRUCTION  
SET ARCHITECTURE

# Objectives

- Understand the factors involved in instruction set architecture design
- Gain familiarity with memory addressing modes
- Understand the concepts of instruction-level pipelining and its effect upon execution performance

# MARIE

- This is the format of a MARIE instruction:



- The fundamental MARIE instructions are:

| Instruction Number |     | Instruction | Meaning                                                                |
|--------------------|-----|-------------|------------------------------------------------------------------------|
| Bin                | Hex |             |                                                                        |
| 0001               | 1   | Load X      | Load the contents of address X into AC.                                |
| 0010               | 2   | Store X     | Store the contents of AC at address X.                                 |
| 0011               | 3   | Add X       | Add the contents of address X to AC and store the result in AC.        |
| 0100               | 4   | Subt X      | Subtract the contents of address X from AC and store the result in AC. |
| 0101               | 5   | Input       | Input a value from the keyboard into AC.                               |
| 0110               | 6   | Output      | Output the value in AC to the display.                                 |
| 0111               | 7   | Halt        | Terminate the program.                                                 |
| 1000               | 8   | Skipcond    | Skip the next instruction on condition.                                |
| 1001               | 9   | Jump X      | Load the value of X into PC.                                           |

# Instruction Formats

- Instruction sets are differentiated by the following:
  - Number of bits per instruction
  - Number of explicit operands per instruction
  - Types of operations (Load, Store, Add, Multiply, ...)
  - Type and size of operands

# Instruction Formats

- In designing an instruction set, consideration is given to:
  - Instruction length (bits)
  - Number of operands: Ex: **1 - Load x** , **2 - Add r1, r2, r3**
  - Number of addressable registers
  - Memory organization
    - Whether byte- or word addressable
  - Addressing modes
    - Direct, Indirect or Indexed

# What is Endianness?

## Big-Endian vs Little-Endian

- If my computer reads **bytes** from left to right, and your computer reads from right to left, we're going to have issues when we need to communicate
- If we have a two-byte integer, the integer may be stored so that the least significant byte is followed by the most significant byte or vice versa.
  - In ***little-endian*** machines, the least significant byte is followed by the most significant byte.
  - In ***Big-endian*** machines store the most significant byte first (at the lower address).

# Instruction Formats – 32 bit word

- As an example, suppose we have the hexadecimal number **0x12345678**
- The big-endian and little-endian arrangements of the bytes are shown below

| Address →     | 00 | 01 | 10 | 11 |
|---------------|----|----|----|----|
| Big Endian    | 12 | 34 | 56 | 78 |
| Little Endian | 78 | 56 | 34 | 12 |

## 5.2 Instruction Formats

- A larger example: A computer uses 32-bit integers. The values 0xABCD1234, 0x00FE4321, and 0x10 would be stored sequentially in memory, starting at address 0x200 as here.

| Byte Order |            |               |
|------------|------------|---------------|
| Address    | Big Endian | Little Endian |
| 0x200      | AB         | 34            |
| 0x201      | CD         | 12            |
| 0x202      | 12         | CD            |
| 0x203      | 34         | AB            |
| 0x204      | 00         | 21            |
| 0x205      | FE         | 43            |
| 0x206      | 43         | FE            |
| 0x207      | 21         | 00            |
| 0x208      | 00         | 10            |
| 0x209      | 00         | 00            |
| 0x20A      | 00         | 00            |
| 0x20B      | 10         | 00            |



## 5.4 Addressing (6 of 6)

- For the LOAD 800, what value is loaded into the accumulator for each addressing mode?

| Memory |        |
|--------|--------|
| 0x800  | 0x900  |
| ...    |        |
| 0x900  | 0x1000 |
| ...    |        |
| 0x1000 | 0x500  |
| ...    |        |
| 0x1100 | 0x600  |
| ...    |        |
| 0x1600 | 0x700  |

R1 0x800

| Mode      | Value Loaded into AC |
|-----------|----------------------|
| Immediate | 0x800                |
| Direct    | 0x900                |
| Indirect  | 0x1000               |
| Indexed   | 0x500                |

# Addressing Modes

- *Immediate addressing* is where the data is part of the instruction.
- *Direct addressing* is where the address of the data is given in the instruction.
- *Register addressing* is where the data is located in a register.
- *Indirect addressing* gives the address of the data in the instruction.
- *Register indirect addressing* uses a register to store the address of the data.

# Immediate addressing

## C Example:

c

 Copy code

```
#include <stdio.h>

int main() {
    int result = 10 + 5; // Immediate values 10 and 5 are directly used
    printf("Result: %d\n", result);
    return 0;
}
```

## Python Example:

python

 Copy code

```
result = 10 + 5 # Immediate values 10 and 5 are directly used
print("Result:", result)
```

# Direct addressing

```
#include <stdio.h>

int main() {
    int x = 10; // Variable declaration and initialization
    printf("Value of x: %d\n", x); // Direct addressing to access the value of x

    x = 20; // Directly modifying the value of x
    printf("New value of x: %d\n", x);

    return 0;
}
```

# Indexed Addressing

```
int a = 5; // Assume 'a' is stored in a register by the compiler  
int b = 10; // Assume 'b' is also stored in a register  
int c = a + b; // The operation happens using register-stored values
```

# Indexed Addressing

```
int array[] = {10, 20, 30, 40};  
int index = 2;  
int value = array[index];  
printf("%d\n", value); // Output: 30
```

Python:

list = {10, 20, 30, 40}

index = 2

Value = list[index]

Print(value)

## 5.4 Addressing (6 of 6)

- For the LOAD 800, what value is loaded into the accumulator for each addressing mode?

| Memory |        |
|--------|--------|
| 0x800  | 0x900  |
| ...    |        |
| 0x900  | 0x1000 |
| ...    |        |
| 0x1000 | 0x500  |
| ...    |        |
| 0x1100 | 0x600  |
| ...    |        |
| 0x1600 | 0x700  |

R1 0x800

| Mode      | Value Loaded into AC |
|-----------|----------------------|
| Immediate | 0x800                |
| Direct    | 0x900                |
| Indirect  | 0x1000               |
| Indexed   | 0x500                |

# Addressing Modes

- *Indexed addressing* uses a register (implicitly or explicitly) as an offset, which is added to the address in the operand to determine the effective address of the data.
- *Based addressing* is similar except that a base register is used instead of an index register.
- The difference between these two is that an index register holds an offset relative to the address given in the instruction, and a base register holds a base address where the address field represents a displacement from this base.



# Indirect Addressing

```
#include <stdio.h>

int main() {
    int x = 10;           // Direct value
    int *ptr = &x;        // Pointer holding the address of x

    printf("Value of x: %d\n", x);           // Direct access
    printf("Value of x using pointer: %d\n", *ptr); // Indirect access (dereferencing)

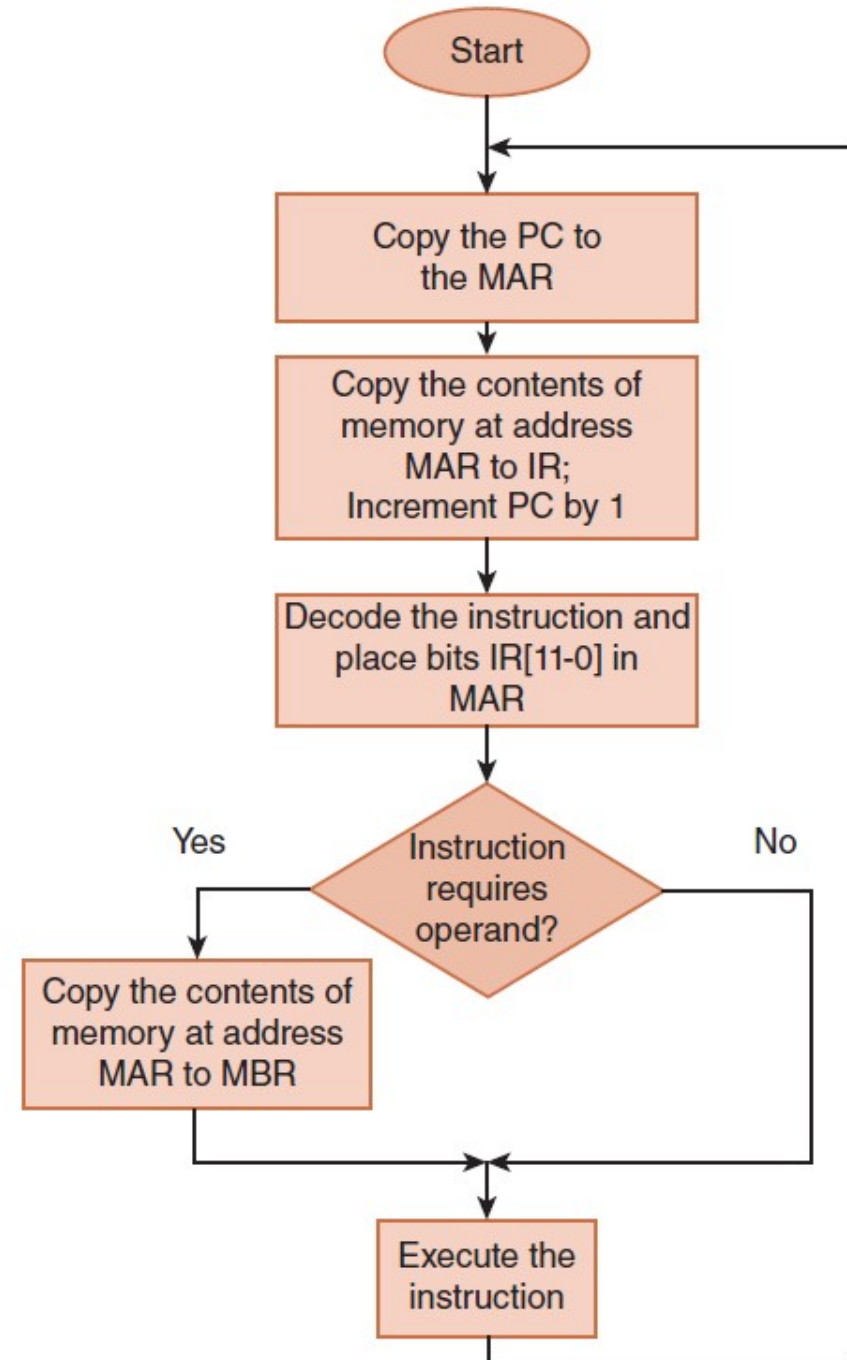
    *ptr = 20; // Modifying x indirectly through pointer
    printf("New value of x: %d\n", x);

    return 0;
}
```

# Instruction Pipelining

- Some CPUs divide the fetch-decode-execute cycle into smaller steps
- These smaller steps can often be executed in parallel to increase throughput
- Such parallel execution is called *instruction pipelining*
- Instruction pipelining provides for *instruction level parallelism* (ILP)

# Instruction Processing



# Instruction Pipelining

- Suppose a fetch-decode-execute cycle were broken into the following smaller steps:
  1. Fetch instruction
  2. Decode opcode
  3. Calculate effective address of operands
  4. Fetch operands
  5. Execute instruction
  6. Store result
- Suppose we have a six-stage pipeline. S1 fetches the instruction, S2 decodes it, S3 determines the address of the operands, S4 fetches them, S5 executes the instruction, and S6 stores the result.

| S1                   | S2               | S3                                               | S4                | S5      | S6              |
|----------------------|------------------|--------------------------------------------------|-------------------|---------|-----------------|
| Fetch<br>Instruction | Decode<br>opcode | Calculate<br>effective<br>address of<br>operands | Fetch<br>operands | Execute | Store<br>result |

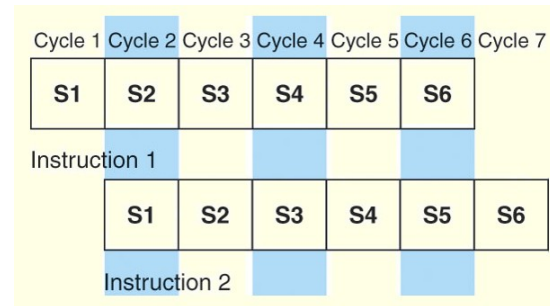
# None-Pipelining Processor

| CYCLE 1       | CYCLE 2 | CYCLE 3 | CYCLE 4 | CYCLE 5 | CYCLE 6 | CYCLE 7       | CYCLE 8 | CYCLE 9 | CYCLE 10 | CYCLE 11 | CYCLE 12 | CYCLE13       | CYCLE14 | CYCLE 15 | CYCLE 16 | CYCLE 17 | CYCLE 18 |
|---------------|---------|---------|---------|---------|---------|---------------|---------|---------|----------|----------|----------|---------------|---------|----------|----------|----------|----------|
| S1            | S2      | S3      | S4      | S5      | S6      | S1            | S2      | S3      | S4       | S5       | S6       | S1            | S2      | S3       | S4       | S5       | S6       |
| INSTRUCTION 1 |         |         |         |         |         | INSTRUCTION 2 |         |         |          |          |          | INSTRUCTION 3 |         |          |          |          |          |

- . Pipelining typically reduces the processor's cycle time and increases the throughput of instructions
- . A non-pipelined processor executes only a single instruction at a time
- . Pipelining doesn't reduce the time it takes to complete an instruction; instead it increases the number of instructions that can be processed simultaneously ("at once") and reduces the delay between completed instructions (called 'throughput')

# Instruction Pipelining

- For every clock cycle, one small step is carried out, and the stages are overlapped.



S1. Fetch instruction.  
S2. Decode opcode.  
S3. Calculate effective  
address of operands.

S4. Fetch operands.  
S5. Execute.  
S6. Store result.

# Is processor pipelining the same as multi-core processor?

No, **processor pipelining** and **multi-core processors** are not the same, though both aim to improve performance in computing. **Processor Pipelining:**

## 1. Definition:

1. Pipelining is a technique used within a **single processor core** to execute multiple instructions simultaneously by breaking down the execution process into stages.
2. Each stage in the pipeline performs part of the instruction (e.g., fetching, decoding, executing, etc.).

## 2. Goal:

1. To improve the instruction throughput (number of instructions completed per unit time).

## 3. How It Works:

1. Different stages of multiple instructions are processed at the same time, like an assembly line in a factory.
2. For example, while one instruction is being executed, another is being decoded, and yet another is being fetched.

## 4. Example:

1. A 5-stage pipeline might have stages like Fetch, Decode, Execute, Memory Access, and Write Back.
2. Instructions flow through these stages concurrently.

## 5. Limitations:

1. Performance can be affected by hazards (e.g., data dependencies, branch mispredictions).
2. It's still limited to the execution of a single stream of instructions.

# Is processor pipelining the same as multi-core processor?

## Multi-Core Processors:

### 1. Definition:

1. A multi-core processor contains two or more independent processing units (cores) within a single chip.
2. Each core can execute its own instructions independently or in parallel with others.

### 2. Goal:

1. To enable true parallel execution of multiple instruction streams or threads.

### 3. How It Works:

1. Each core operates as a separate processor, capable of handling its own tasks.
2. Software must be written to take advantage of multiple cores (multi-threading).

### 4. Example:

1. A quad-core processor has four cores that can potentially execute four different tasks simultaneously.

### 5. Advantages:

1. Handles multiple tasks (threads or processes) concurrently.
2. Improves performance for multi-threaded applications like gaming, video rendering, and AI computations.

### 6. Limitations:

1. Benefits depend on whether the software is optimized for multiple cores.
2. Not all tasks can be parallelized effectively



# Is processor pipelining the same as multi-core processor?

## Key Differences:

| Feature             | Pipelining                                 | Multi-Core Processors                      |
|---------------------|--------------------------------------------|--------------------------------------------|
| Focus               | Speeding up the execution of a single task | Handling multiple tasks simultaneously     |
| Hardware            | Uses stages within a single core           | Uses multiple independent cores            |
| Concurrency         | Processes parts of multiple instructions   | Executes multiple instruction streams      |
| Software Dependency | Works for all software                     | Benefits depend on multi-threaded software |

## Example Analogy:

- **Pipelining:** Like an assembly line in a factory where multiple workers perform different steps on a single product.
- **Multi-Core:** Like having multiple assembly lines in the factory, each capable of producing its own product simultaneously.

Both pipelining and multi-core designs are crucial for modern processors, but they solve different problems in computing performance.